

CS 219 Operating Systems

Spring 2025-26

End-semester Exam (45 marks)



Name: _____ Roll No.: _____

Q1 (10)	Q2 (7)	Q3 (11)	Q4 (10)	Q5 (7)	Total (45)

Q1. The following are characteristics of a file system and a disk device.

block size = 1024 bytes

address of block = 4 bytes (to store block no.)

inode size = 32 bytes

size of disk = 8192 blocks

#inodes supported = 128

No support for log-based transactions

(a) Assume a disk has one boot and one superblock. Draw a figure to show the layout of the file system objects on the disk. Mention the start block number of each file system object on the disk.

[3 marks]

Boot Block	Super Block	Inode Bitmap	Data Bitmap	Inode Table	Data Blocks
(1 Block)	(1 Block)	(1 Block)	(1 Block)	(4 Blocks)	(8184 Blocks)
0	1	2	3	4	8 ... 8191

Inode Bitmap:

- Total Inodes = 128 => 128 bits required.
- Since 1 block is 1024 bytes (8192 bits), the bitmap fits into **1 block**.

Data Bitmap:

- Total disk blocks = 8192 => 8192 bits required.
- Since 1 block is 1024 bytes (8192 bits), the bitmap fits into **1 block**.

Inode Table:

- Total inodes = 128; Size of each inode = 32 bytes.
- Total size = $128 \times 32 = 4096$ bytes.
- Size in blocks = $4096/1024 = 4$ **blocks**.

(b) Assuming the inode structure has an INDIRECT pointer for datablocks, [3 marks]

- what is the maximum size of a file supported by the file system?
- what is the maximum size of a file supported by the file system on **this** disk?



Assuming Inode contains one INDIRECT pointer.

Addresses per Indirect Block: Block Size / Address Size = 1024 bytes / 4 bytes = 256 addresses

Max Blocks per File: 256 blocks (via the indirect pointer)

Max File Size: 256 blocks × 1024 bytes = **262,144 bytes (256 KB)**

Max size supported by the file system: 256 KB [1.5 marks]

Even though the disk has 8,184 data blocks available, the file system's use of a single indirect pointer limits any individual file to only 256 blocks. Therefore, the limit remains the same for both.

Max size supported on this disk: 256 KB [1.5 marks]

(c) Consider the following filename path **/home/data/keytoworldpeace.txt**

If the maximum number of files/directories in a directory is 32 and the size of each directory entry is 64 bytes, what is the maximum number of disk fetches that will be needed to resolve the path to its inode number? Assume the inode of the root is known. [2 marks]

Maximum disk read operations = read inode block for every directory/file in path + two blocks to read each sub-directory/file entry
$$= 1 \times 3 + 2 \times 3 = \mathbf{9 \text{ disk reads}}$$

If the inode of the root is considered to be in-memory then the answer is **8 disk reads**.



(d) On a scan of the disk, **fsck** is reporting errors on file **foo.txt**. Using the specifics of the file system listed above, discuss **three different** types of situations in which this would have happened.

The **three** situations should be described using the **values** of variables/objects of the file system.

[2 marks]

Example situations ...

- 1) A data block in the inode of foo.txt is not set in bitmap.
- 2) Filesize of foo.txt is not equal to the allocated blocks for foo.txt.
- 3) The data block used for foo.txt is used by some other files.
- 4) Check sum of a data block belonging to foo.txt is failing

Any relevant conditions mentioning **intersection** with the file foo.txt

At least two reasons should be related to foo.txt. - 2 marks



Q2. (a) Consider the following code segment —

```
int fd1, fd2, fd3;
char buf[500];

fd1 = open("/home/worldpeace.txt", O_RDWR);
fd2 = open("/home/worldpeace.txt", O_RDWR);
fd3 = dup(fd1);
fd4 = dup2(fd1, 8);

read(fd1, buf, 2000);
read(fd2, buf, 5000);
read(fd3, buf, 2000);
lseek(fd4, 10, SEEK_SET);
read(fd1, buf, 10);
```

Draw a picture to show all the file system objects/states in the kernel that is involved in the above file operations, their connections, and annotate them with the information and values they store.

Assume these are the only file operations in the program, and the page size/disk block size is 4096 bytes.

[3 marks]

After the syscalls `open(...)`, `open(...)`, `dup(...)`, `dup2(...)`:

Diagram with the following objects: **2 marks**

1. File descriptor table of the process: 0.5 marks

Each `fd` should be represented using a different offset in the `fd` table, where `fd4 = 8` always. Assuming that at the start of the process only file descriptors for `stdin`, `stdout` and `stderr` are used, the indices for `fd1`, `fd2`, `fd3` and `fd4` should be 3, 4, 5 and 8 respectively. Answers with `fd1`, `fd2` and `fd3` as 0, 1 and 2 respectively are also considered given that the assumption of the student is clearly stated.

2. File structs: 1 mark

There should be a total of two `struct file`:

`struct file 1`: pointed by `fd1`, `fd3`, and `fd4` with `ref_cnt = 3` and `offset = 0`.

`struct file 2`: pointed by `fd2` with `ref_cnt = 1` and `offset = 0`.

3. `inode`: 0.5 marks

One `inode` pointed to by `struct file 1` and `struct file 2` with `ref_cnt = 2`.

Before the first `read(...)` there are no cached blocks.

After the syscalls `read(...)`, `read(...)`, `lseek(...)`, `read(...)`:

The following state update needs to be mentioned: **1 mark**

1. offset of `struct file 1` updated to 20 and offset of `struct file 2` updated to 5000.
2. Total number of cached blocks updated to 2.

Note: The buffer size is mentioned as 500 in the problem. Answers that point out that a memory error can happen due to insufficient buffer size are also considered. Answers that assume that the total number of bytes read will be capped by the buffer size are not considered.



(b) Students of CS219 are concerned about the file system performance and are in mission mode to do something about it. Propose **two** ideas/designs for the same. You can change any components of the file system path and its management actions. Propose two experiments that will demonstrate the potential improvement using **file system** metrics. File read time in a process is a user space application metric and not an under-the-hood file system operations-related metric. **[4 marks]**

Ideas: **2 marks** (1 + 1)

Any two ideas including but not limited to prefetching, block deduplication, metadata caching, block compression, read-ahead caching, ...

Experiments: **2 marks** (1 + 1)

Experiments that may demonstrate improvement by the suggested ideas using any file system performance metric: number of disk I/O operations, average block latency, etc.

Note:

1. Even if the idea targets another goal (e.g., consistency, fault tolerance), marks are given if the experiment is sound and stated clearly.
2. Experiments need to mention the metric being measured, the type of workload, and the expected results.



Q3. (a) Write pseudocode for the producers-consumers synchronization problem using **condition variables** and **mutexes**. Note that several producers add to the same queue, and several consumers consume from it. Producers cannot add objects to the queue while a consumer is accessing it; consumers cannot remove an object while a producer or another consumer is accessing the queue.

Write pseudo-code for the **produce**, **consume**, **sleep**, and **wakeup** functions and define variables as needed. **[4 marks]**

Assume the following functions exist —

add_to_queue(); // adds a new object to the shared queue

remove_from_queue(); // removes an object from the queue

Mutex m;

Cond_var Sp = MAX_SIZE;

Cond_var Sc = 0;

<pre>produce(Item) { lock(&m); while (Sp <= 0) { sleep(&Sp, m); } Sp--; Sc++; add_to_queue(item); wakeup(&Sc); unlock(&m); }</pre>	<pre>consume() { lock(&m); while (Sc <= 0) { sleep(&Sc, m); } Sc--; Sp++; remove_from_queue(); wakeup(&Sp); unlock(&m); }</pre>
<pre>sleep(c, m) { // P = current process P->state = WAITING; P->chan = c; unlock(&m); sched(); lock(&m); }</pre>	<pre>wakeup(c) { // Iterate over all processes P in the process table for (all processes) { if (P->chan == c) { P->state = READY; P->chan = 0; } } }</pre>

Grading — Each function is worth 1 mark

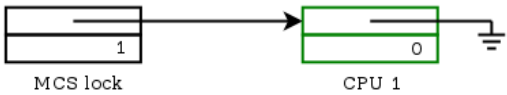
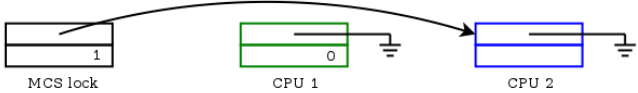
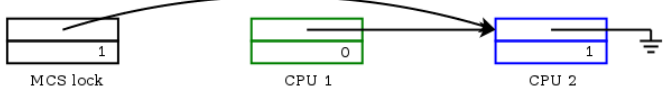


(b) MCS (Mellor-Crummey and Scott) spinlocks aim to be CPU cache friendly and, importantly, **fair** — spinlocks are released to CPUs in the order that CPUs attempt to obtain the lock.

MCS locks use per-CPU spinlock structures and a global MCS lock.

The following is an example of a kernel structure for MCS lock implementation.

```
struct mcs_spinlock {  
    struct mcs_spinlock *next;  
    int locked; /* 1 if lock acquired */  
};  
struct mcs_spinlock mcs_lock[NCPU+1]; // 1 for global, rest for each CPU
```

(i) Global MCS lock is free 	
	
	The lock status is always with the global MCS lock. Per-CPU spinning is on the local lock variable value. Local lock variable = 0 indicates the lock is with CPU. MCS lock always points to the tail of the waiting list. Multiple CPUs may check/want the lock concurrently.

(i) Write pseudo-code for actions on a CPU attempting to obtain a spinlock? **[2 marks]**

(iii) Write pseudo-code for actions on a CPU when releasing a spinlock? **[1 mark]**

(iii) Which operations need to be synchronized? Why? **[1 mark]**



mcs[0] is the global lock

mcs[i] are the local locks (one per cpu)

```
lock(i) // assuming CPU i is attempting to grab the spinlock
{
    temp = &mcs[i];

    // atomic exchange to set tail pointer to CPU i
    // mcs[0].next points to mcs[i]
    // and temp gets prev mcs[0].next pointer
    prev = atomic_xchg(mcs[0].next, temp);

    if (prev != NULL) {
        mcs[i].locked=1;
        prev->next = &mcs[i];
        while(mcs[i].locked);    // spin on local per-cpu variable
    }
    else {                        // got lock
        mcs[i].locked=0;
        mcs[0].locked=1;
        mcs[0].next = &mcs[i];
    }
}

unlock(i)
{
    temp &mcs[i];
    if(mcs[i].next == NULL) {    // no waiting CPUs
        // check if next of global lock is CPU i, & if so set it to NULL
        if (cmp_xchg(mcs[0].next, NULL, temp)) {
            mcs[0].locked = 0;
            return;
        }
    }
    // a new CPU awaits or is being added
    while(mcs[i].next!=NULL);

    mcs[i].next->locked = 0;
    mcs[i].next=NULL;
}
```




(c) Consider three threads of concurrent execution. Each thread executes a set of instructions A, B, and C, suffixed by the thread identifier. Within a given thread, the A, B, and C instructions are issued in the order as shown.

The following happens-before relationships need to be maintained during execution.

A.1 before A.2 and A.3	B.1 after B.3
C.2 before C.1 and C.3	C.1 before C.3

The execution order of the other instructions can be any order. Assume three semaphores, S1, S2, and S3. Complete the following pseudo-code by adding instructions in the empty spaces to enforce the happens-before/after relationships mentioned above. [3 marks]

// start code here

S1.init(0); S2.init(0); S3.init(0);

Thread 1	Thread 2	Thread 3
A.1 up(S1) up(S1) down(S2) B.1 down(S3) C.1 up(S1)	down(S1) A.2 up(S3) B.2 C.2 up(S3) up(S3)	down(S1) A.3 down(S3) B.3 up(S2) down(S3) down(S1) C.3

// start code here

S1.init(0); S2.init(0); S3.init(0);

Thread 1	Thread 2	Thread 3
A.1 up(S1) down(S3) B.1 C.1 up(S2)	down(S1) A.2 B.2 C.2 up(S2)	down(S2) A.3 B.3 up(3) down(S2) C.3

Instructions — A and B are 0.5 marks each, C and D are 1 marks each.
Penalty of -0.5 if semaphores are not initialized.



Q4. State TRUE or FALSE and justify.

[10 x 1 = 10 marks] No correct justification, no marks.

- a. The **nxt3.2** filesystem can always read files stored on a disk managed by the **nxt3.1** file system.

False, mention VFS not available or read file operation not matching across filesystems. (1)

True, then need to mention both VFS read operation available and read permission is set. (1)

0.5 if False and read permission is the reason.

b.

```
p = 23;
fork();
p = p+1;
printf("%d ",p);
```

The output of the pseudo-code with the **copy-on-write** optimization is — **24 25**

False, the variable p will have value 23 after fork() for both parent and child. Even with CoW the moment any change appears both will be provided their own physical pages. Thus, both will execute on p=23. So final values for both will be 24. Need to mention reason and answer 24 24. (1)
Not mentioning one of them gives 0.5.

- c. A page is swapped to disk only if the process that the page belongs to is under memory pressure.

False, if any other process is under memory pressure, pages of this process may also be swapped. Reasons can also be swapping policy or pages not accessed for a long time being swapped automatically. (1)
Other valid reasons have been considered.

- d. Files managed by the **procfs** filesystem in the **/proc** directory can be read even if they are zero bytes in size.

False, and the reason must mention that files here are written on-fly by OS as the read commands are mapped to specific kernel functions.(1)
Not mentioning OS or kernel will have 0.5 deducted. Other reasons like file is always read have been awarded 0.



- e. Assuming no page faults, the MMU hardware sets the page-accessed bit of a PTE during a memory access, not the OS.

True, MMU handles the memory access unless Page Fault occurs when OS steps in.(1)

Not mentioning unless page fault or just writing the statement back like MMU handles the memory access have been awarded 0.5.

- f. An optimized `exec` system call can return to user space by allocating only three extra pages for the new process. Assuming a two-level page table.

False. A two level page table will require at least 2 pages. Additional pages are also strictly necessary to map the code, data, and stack segments.

True will be also considered if EXPLICITLY mentioned/assumed a single page can contain code, data, and stack.

- g. Kernel threads of execution can *deadlock* **only** if the interrupts are not disabled when holding a spinlock.

False. Consider the case of multi-core processor

CPU 1 (Thread A): Disables interrupts and acquires Spinlock X

CPU 2 (Thread B): Disables interrupts and acquires Spinlock Y

CPU 1 (Thread A): Attempts to acquire Spinlock Y (and begins spinning)

CPU 2 (Thread B): Attempts to acquire Spinlock X (and begins spinning)

True will be also considered if EXPLICITLY assumed a single-core processor.

- h. An OS can execute only one instance of the scheduler at any point in time.

False. In a multi-core processor, each core can run its own instance of scheduler.

- i. With the write-through page caching technique — which writes data simultaneously to both the cache and the underlying storage, there is no problem of file system consistency.

False. Writing a file is not a single, atomic action. It requires multiple physical disk updates for writing the actual data, updating the bitmap and updating the inode. If a power failure occurs halfway through these steps, the file system will be left inconsistent.

- j. `kswapd`, the kernel swapper daemon (a kernel background process), is not needed if the kernel uses lazy mapping of malloc'ed pages.

False. Lazy mapping only delays when physical memory is allocated. As programs run and use their malloc'ed memory, the physical memory will eventually fill up. When the system runs low on free physical memory, `kswapd` will be required.



Q5. (a) Consider an OS that operates with a two-level page table.

The TLB hit rate is 10%, and physical memory access latency is 100us, Average page fault handling time (to traverse the page table, generate the fault, and fix the page table) is 1ms, and the number of page faults are 5% of all accesses after TLB miss. Assume the TLB look-up time is negligible.

What is the average time it takes to read/access a virtual address? **[3 marks]**

average time = $0.1 * 100 + 0.9 * 0.05 * 1000 + 0.9 * 300 = 10 + 45 + 270 = 325\text{us}$

(b) Write pseudo-code for a page fault handler's logic that identifies and handles memory accesses to pages **swapped** to disk. Assume that the page fault handler has no input arguments when invoked; i.e., it must determine all its execution context after invocation. **[4 marks]**

```
fva = lcr2();           // read CR2 to store faulting virtual address
pgdir = getpgdir(myproc()); // find process and then find page table of process issuing address

pte = walkpgdir(pgdir, fva); // walk pgdir and find pte

// determine if page is swapped out
if( (pte_flags(pte) & PTE_VALID) && !(pte_flags(pte) & PTE_P) )
{
    // find location on disk
    diskblock = swap_location(pte);

    // allocate a page in memory
    kva = kalloc();

    // copy contents from disk to memory
    readpage(diskblock, kva);

    // add mapping to to page table
    mappages(pgdir, fva, V2P(kva));

    // update PTE's present and valid mapping
    *pte = *pte | PTE_P | PTE_V | PTE_U ;
}
else
{
    ...
    // non-swap page handling
}
```

Rough work



Rough work

